

Факультет компьютерных наук

14 ноября 2026 г.

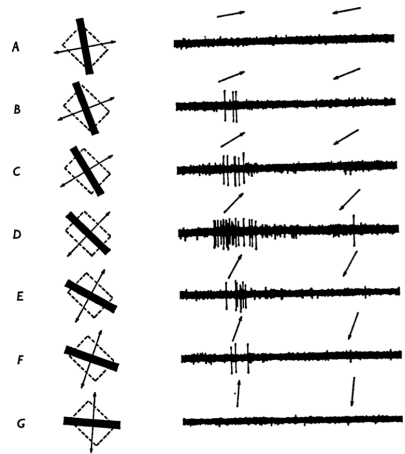
Сверточные нейронные сети

Жарковский Дмитрий Андреевич
garkovski_dmitri@mail.ru

Зрение и нейроны

Как биология подсказала архитектуру сети

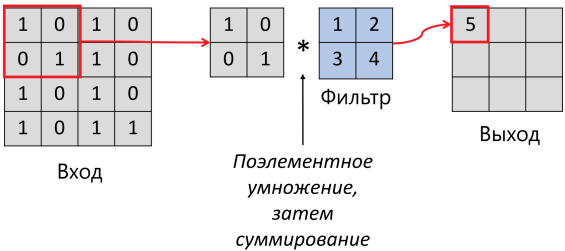
- В опытах на зрительной коре кошки регистрировали отклик отдельных нейронов на полоски света под разным углом;
- каждый нейрон сильно реагирует только на **свой** угол наклона линии — и почти не реагирует на остальные;
- то есть нейрон — это детектор конкретного локального паттерна в небольшой области поля зрения.



Источник изображения и статья: https://www.ncbi.nlm.nih.gov/pmc/articles/PMC1557912/

Свёртка (Convolution)

Как считается один выходной пиксель



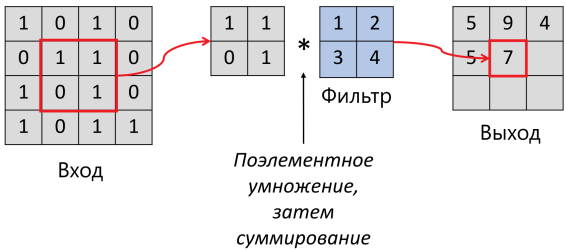
- берём окно входа размером с фильтр (kernel);
- умножаем значения поэлементно;
- складываем все произведения — получаем одно число выхода.

Источник изображения:

https://github.com/hse-ds/iad-deep-learning/blob/master/2022/lectures/lecture02-convnets.pdf

Свёртка (Convolution)

Окно скользит по всему изображению



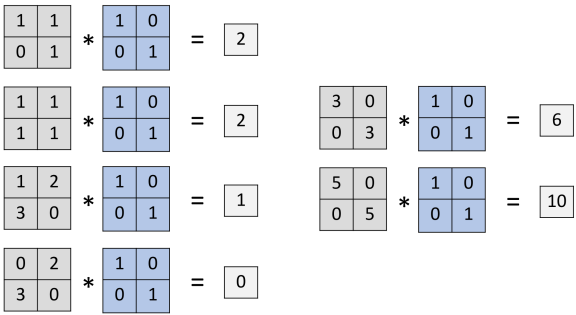
- фильтр сдвигается на новую позицию;
- та же операция повторяется для каждого положения;
- так заполняется вся выходная карта признаков (*feature map*).

Источник изображения:

https://github.com/hse-ds/iad-deep-learning/blob/master/2022/lectures/lecture02-convnets.pdf

Свёртка (Convolution)

Чем больше совпадение — тем больше отклик



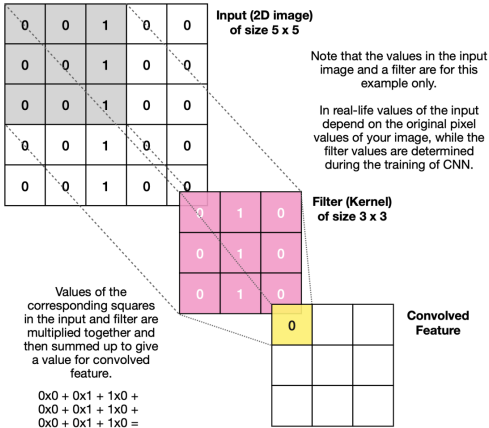
- фильтр (синего цвета) — один и тот же во всех примерах;
- чем больше совпадение, тем больше выход;

Источник изображения:

https://github.com/hse-ds/iad-deep-learning/blob/master/2022/lectures/lecture02-convnets.pdf

Свёртка (Convolution)

Фильтр как детектор паттерна



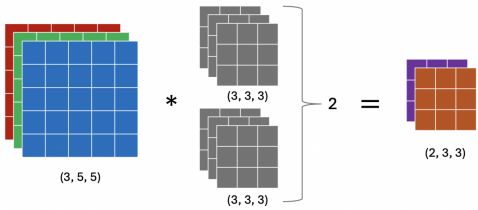
- здесь фильтр — “столбик” из единиц, то есть детектор **вертикальной линии**;
- на вход подана вертикальная полоса из единиц;
- итоговое значение получается суммой поэлементных произведений.

Источник изображения:

https://www.linkedin.com/pulse/one-minute-overview-deep-convolutional-neural-networks-dobilas

Свёртка для нескольких каналов

Цветные изображения и несколько фильтров



- вход — 3-канальное (RGB) изображение 5×5;
- у каждого фильтра глубина равна числу входных каналов;
- два разных фильтра — два выходных канала.

Источник изображения: https://habr.com/ru/articles/976410/

Почему свёртка эффективнее полносвязного слоя

Пример: изображение MNIST 28×28

- Полносвязный слой “каждый с каждым” между входом и выходом такого же размера:

$$28 \cdot 28 \;=\; 784 \text{ входов}, \qquad 784 \times 784 + 784 \approx 615\,000 \text{ параметров}.$$

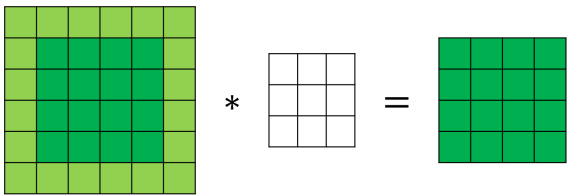
- Один сверточный фильтр 3×3 (плюс смещение) — всего

$$3 \cdot 3 + 1 = 10 \text{ параметров},$$

  и это число **не растёт** с размером изображения;
- экономия достигается за счёт *разделяемых весов* (*weight sharing*) — один и тот же маленький фильтр используется во всех положениях картинки.

Отступ (Padding)

Зачем нужна рамка вокруг изображения



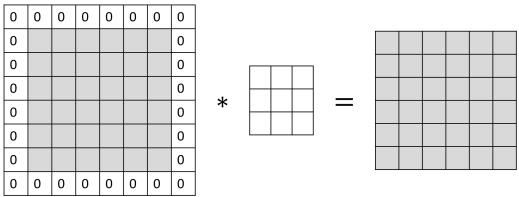
- без рамки свёртка **уменьшает** размер картинки на каждом слое;
- пиксели на краю участвуют в свёртке меньше раз, чем центральные;
- padding добавляет искусственную рамку вокруг входа, чтобы сохранить размер и информацию с краёв.

Источник изображения:

https://github.com/hse-ds/iad-deep-learning/blob/master/2022/lectures/lecture04-convnets-optimization.pdf

Zero padding

Рамка из нулей



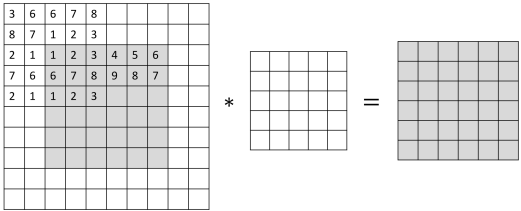
- самый распространённый способ;
- по краям добавляются пиксели со значением 0;
- просто и не искажает статистику самого изображения слишком сильно.

Источник изображения:

https://github.com/hse-ds/iad-deep-learning/blob/master/2022/lectures/lecture04-convnets-optimization.pdf

Reflection padding

Рамка — зеркальное отражение



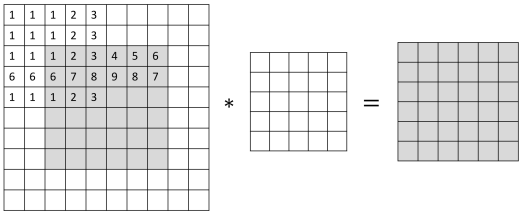
- новые пиксели — зеркальное отражение соседних пикселей изображения;
- не добавляет “искусственных” нулей, поэтому меньше искажает края.

Источник изображения:

https://github.com/hse-ds/iad-deep-learning/blob/master/2022/lectures/lecture04-convnets-optimization.pdf

Replication padding

Рамка — повторение крайних пикселей



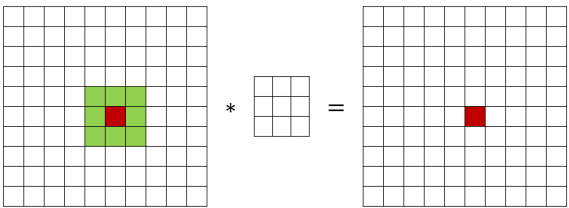
- крайний пиксель просто копируется наружу столько раз, сколько нужно;
- другой способ избежать резкого перепада значений на границе.

Источник изображения:

https://github.com/hse-ds/iad-deep-learning/blob/master/2022/lectures/lecture04-convnets-optimization.pdf

Поле восприятия

От какой части входа зависит один выходной пиксель



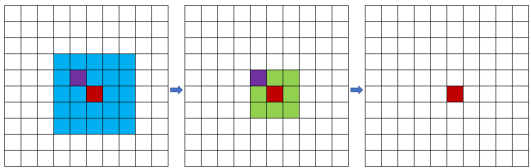
- **Поле восприятия** пикселя выхода — это область входного изображения, которая влияет на его значение;
- после одного слоя свёртки 3×3 поле восприятия каждого выходного пикселя — тоже 3×3.

Источник изображения:

https://github.com/hse-ds/iad-deep-learning/blob/master/2022/lectures/lecture04-convnets-optimization.pdf

Поле восприятия растёт со стекингом слоёв

Ядро 3×3



После n слоёв свёртки 3×3 подряд:

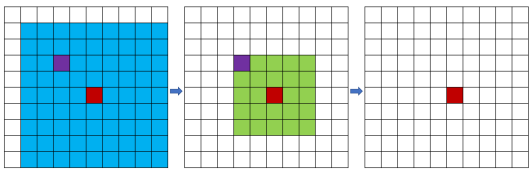
- 1 слой — 3×3;
- 2 слоя — 5×5;
- 3 слоя — 7×7.

Источник изображения:

https://github.com/hse-ds/iad-deep-learning/blob/master/2022/lectures/lecture04-convnets-optimization.pdf

Поле восприятия растёт со стекингом слоёв

Ядро 5×5



После n слоёв свёртки 5×5 подряд:

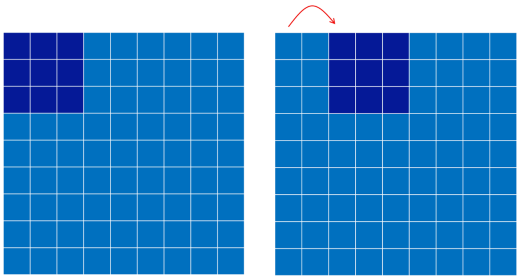
- 1 слой — 5×5;
- 2 слоя — 9×9;
- 3 слоя — 13×13.

Источник изображения:

https://github.com/hse-ds/iad-deep-learning/blob/master/2022/lectures/lecture04-convnets-optimization.pdf

Шаг свёртки (Stride)

Пропускаем пиксели при движении фильтра

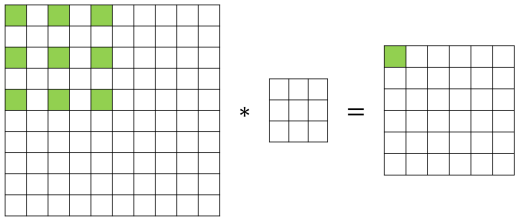


- stride — шаг, с которым фильтр сдвигается по изображению;
- stride > 1 уменьшает размер выходной карты признаков;
- увеличивает поле восприятия быстрее, чем обычная свёртка с шагом 1.

Источник изображения: https://habr.com/ru/articles/976410/

Раздутые свёртки (Dilated convolutions)

Фильтр с “дырками” между элементами



- между элементами фильтра вставляются пропуски (*dilation*);
- фильтр той же формы 3×3 “охватывает” намного большую область входа;
- увеличивает поле восприятия без увеличения числа параметров фильтра.

Источник изображения:

https://github.com/hse-ds/iad-deep-learning/blob/master/2022/lectures/lecture04-convnets-optimization.pdf

Размер выхода свёрточного слоя

Одна формула для высоты и для ширины

$$H_{\text{out}} = \left\lfloor \frac{H_{\text{in}} + 2P - D(K-1) - 1}{S} \right\rfloor + 1$$

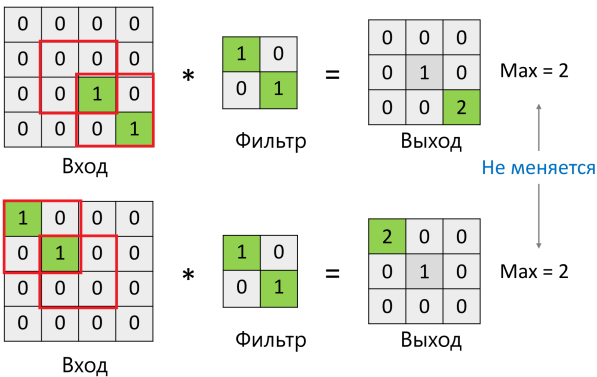
- H_{in} — сторона входа, K — ядро, P — padding, S — stride, D — dilation;
- охват ядра — $D(K-1)+1$ пикселей; при обычной свёртке $D=1$ это просто K;
- из входа с рамкой вычитаем охват, делим на шаг S и округляем вниз;
- прибавляем 1 — первую позицию фильтра; неполный шаг отбрасывается;
- ширина считается так же; каналов на выходе столько, сколько фильтров.

Пример: вход 28×28, ядро 3×3, $P=1$, $S=2$, $D=1$; охват ядра равен 3

$$\left\lfloor \frac{28 + 2 \cdot 1 - 3}{2} \right\rfloor + 1 = \left\lfloor \frac{27}{2} \right\rfloor + 1 = 14.$$

Свёртка инвариантна к сдвигам

Один и тот же паттерн — один и тот же максимум отклика



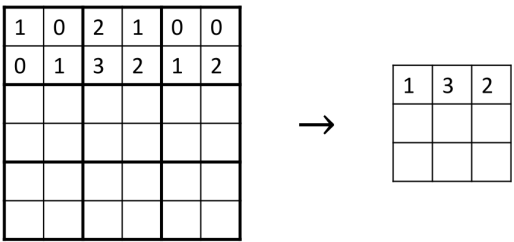
- паттерн на входе сдвинули — карта признаков тоже сдвинулась;
- максимум отклика фильтра при этом **не меняется**;
- поэтому сеть узнаёт один и тот же признак независимо от его положения на картинке.

Источник изображения:

https://github.com/hse-ds/iad-deep-learning/blob/master/2022/lectures/lecture02-convnets.pdf

Pooling

Max pooling 2×2



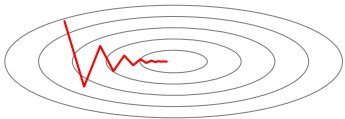
- из каждого окна 2×2 берётся максимальное значение;
- уменьшает размер карты признаков и, тем самым, увеличивает поле восприятия следующих слоёв;
- у pooling-а **нет обучаемых параметров**;
- важно, чтобы поле восприятия последнего сверточного слоя было сравнимо со всей картинкой.

Источник изображения:

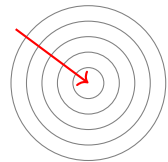
https://github.com/hse-ds/iad-deep-learning/blob/master/2022/lectures/lecture04-convnets-optimization.pdf

Нормировка данных

Почему градиентный спуск чувствителен к масштабу признаков



Не нормированные признаки

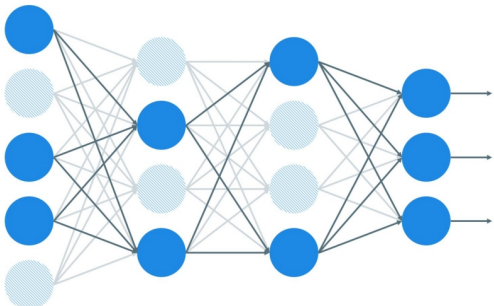


Нормированные признаки

- если один признак измеряется в единицах, а другой — в тысячах, “ландшафт” функции потерь вытягивается в эллипс;
- градиентный спуск начинает “зигзагом” долго спускаться к минимуму вместо прямого пути;
- нормировка (например, вычитаем среднее и делим на стандартное отклонение) делает масштаб признаков сопоставимым — спуск идёт быстрее и устойчивее.

Dropout

Случайное “выключение” нейронов

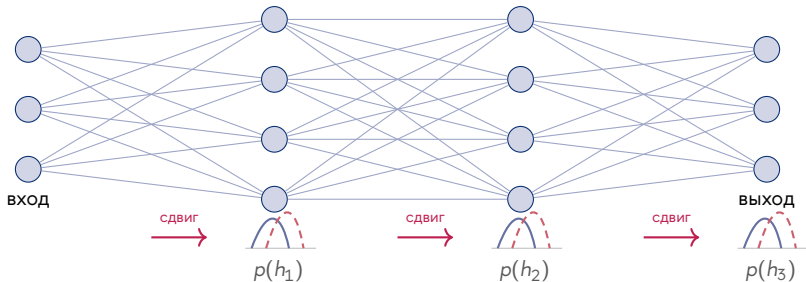


- на каждом шаге обучения нейрон “выключается” с вероятностью p;
- на инференсе все нейроны включены; масштаб компенсируют:
  - домножение на $(1 - p)$ на инференсе;
  - или inverted dropout: $\times \frac{1}{1-p}$ при обучении.

Источник изображения: https://habr.com/ru/companies/mvideo/articles/782360/

Internal Covariate Shift

Распределение входов слоя “плывёт” во время обучения



- слоям приходится заново подстраиваться — обучение замедляется; это и есть *internal covariate shift*.

Batch Normalization

На примере

мини-батч:
объекты $\times$ каналы

1 **Вход** X

	c_1	c_2
x_1	1	1
x_2	2	4
x_3	0	-1
x_4	3	0

2 **Статистики**

	c_1	c_2
μ_B	1,5	1
σ_B^2	1,25	3,5

μ_B, σ_B^2
по каждому каналу

центрируем
и масштабируем:
$\mathbb{E} \approx 0, \mathrm{Var} \approx 1$

3 **Нормировка** $\hat{X}$

$$\hat{x} = \frac{x - \mu_B}{\sqrt{\sigma_B^2 + \varepsilon}}$$

$$\varepsilon = 10^{-6}$$

	c_1	c_2
x_1	$-0{,}45$	0
x_2	0,45	1,60
x_3	$-1{,}34$	$-1{,}07$
x_4	1,34	$-0{,}53$

4 **Масштаб** y

$$y = \gamma\hat{x} + \beta$$

$$\gamma = 2, \beta = 1$$

	c_1	c_2
x_1	0,11	1
x_2	1,89	4,21
x_3	$-1{,}68$	$-1{,}14$
x_4	3,68	$-0{,}07$

обучаемые γ, β
задают нужный масштаб

Инициализация весов

С каких значений начинать обучение

- слишком большие веса — активации и градиенты “взрываются”;
- слишком маленькие — сигнал затухает, градиент почти не доходит до ранних слоёв;
- **Xavier/Glorot** (сигмоида, tanh): сохраняет масштаб сигнала,

$$W \sim \mathcal{N}\left(0,\ \frac{2}{n_{\text{in}} + n_{\text{out}}}\right) \quad \text{или} \quad W \sim U\left(-\sqrt{\frac{6}{n_{\text{in}} + n_{\text{out}}}},\ \sqrt{\frac{6}{n_{\text{in}} + n_{\text{out}}}}\right);$$

- **He** (ReLU): дисперсия примерно вдвое больше — компенсирует зануление половины активаций,

$$W \sim \mathcal{N}\left(0,\ \frac{2}{n_{\text{in}}}\right) \quad \text{или} \quad W \sim U\left(-\sqrt{\frac{6}{n_{\text{in}}}},\ \sqrt{\frac{6}{n_{\text{in}}}}\right).$$

Weight decay и Gradient clipping

Два способа держать веса и градиенты “в рамках”

Weight decay

- к функции потерь добавляется штраф за большие веса (например, $\lambda \sum w^2$);
- сеть “не хочет” делать веса слишком большими без необходимости;
- уменьшает переобучение, помогает обобщению.

Gradient clipping

- если норма градиента превышает порог T, градиент масштабируется:

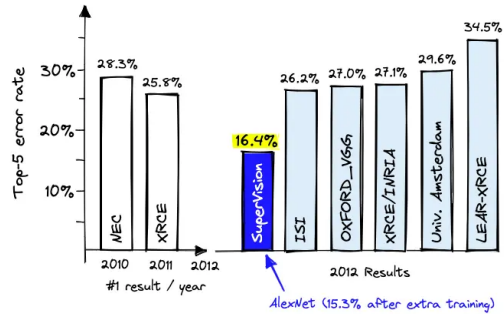
$$g \leftarrow g \cdot \frac{T}{\|g\|}, \quad \text{если } \|g\| > T;$$

- защищает от “взрыва” градиентов и слишком резких шагов обучения.

ImageNet

Соревнование, которое изменило компьютерное зрение

- ImageNet — $> 10^6$ размеченных изображений, 1000 классов;
- ILSVRC — классификация по одному из 1000 классов;
- метрика: *top-5 error* — правильный класс не в топ-5 предсказаний.

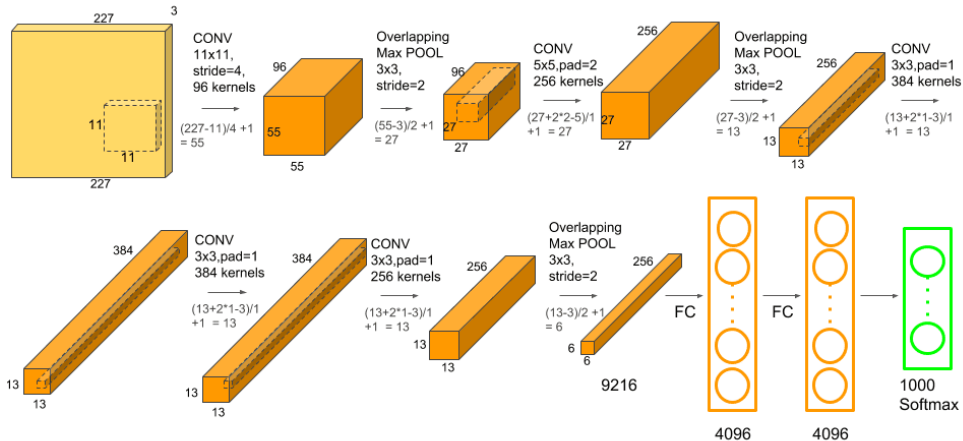


В 2012 году сверточная сеть **SuperVision (AlexNet)** резко обошла все предыдущие подходы — 16.4% ошибки против 25.8% у лучшего результата 2011 года.

Источник изображения: https://www.pinecone.io/learn/series/image-search/imagenet/

AlexNet

Архитектура-победитель ILSVRC-2012



Итоги

Что мы разобрали

- идея свёртки пришла из биологии зрения;
- свёртка — локальный детектор паттерна с разделяемыми весами, экономнее полносвязного слоя;
- padding, stride, dilated convolution и pooling управляют размером выхода и полем восприятия;
- свёртка инвариантна к сдвигу и умеет работать с несколькими каналами.
- нормировка данных и правильная инициализация весов ускоряют обучение;
- weight decay, gradient clipping, dropout и batch normalization помогают обучать глубокие сети быстрее и без переобучения;
- ImageNet и AlexNet — пример того, как эти идеи вместе дали прорыв в компьютерном зрении.

Спасибо за внимание!

Жарковский Дмитрий Андреевич
garkovski_dmitri@mail.ru